

# **Synnepha Technical Assessment Report**

## **AWS Cost Anomaly Detection And Forecasting Assessment**

**Prepared by:**

Eser İnan Arslan

Senior Data Scientist / Machine Learning Engineer

# 1. Objective

The objective of this assessment was to:

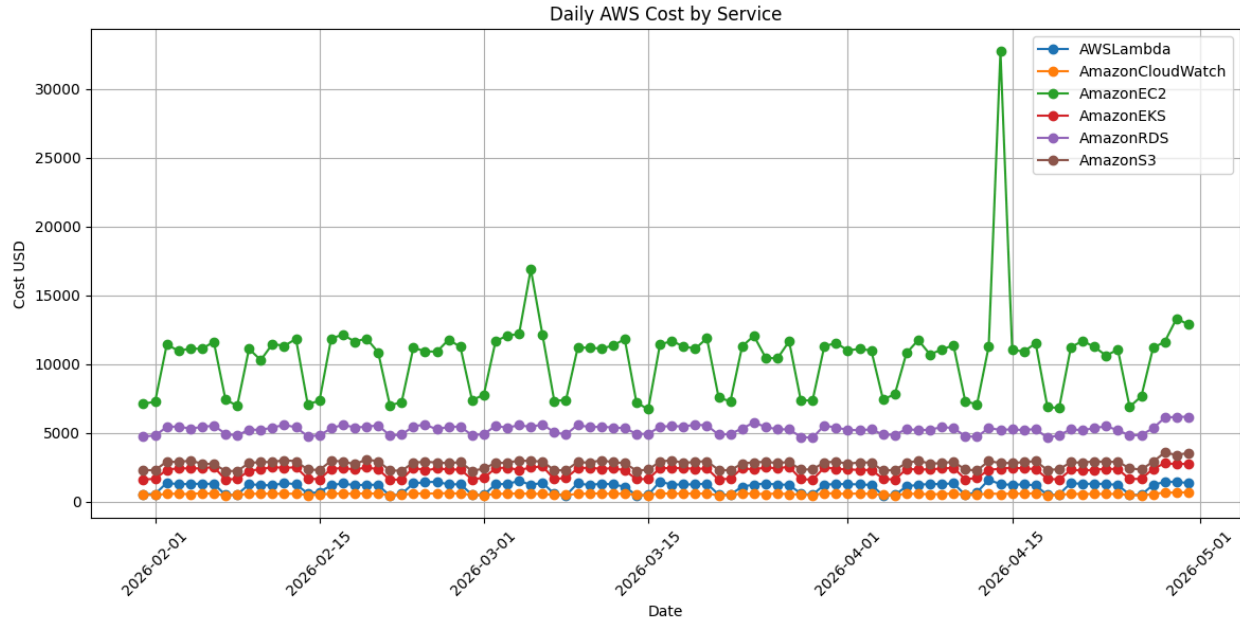
1. Identify candidate AWS cost anomalies.
2. Forecast future AWS costs using historical billing data.
3. Design a production-ready architecture capable of running the detection pipeline daily.
4. Identify important business and data considerations before production deployment.

The solution combines AWS Cost and Usage Report (CUR) data with Kubernetes workload metrics in order to detect unusual cost behaviour and understand potential workload-related drivers behind cost fluctuations.

---

## 2. Exploratory Data Analysis (EDA)

Before building any model, I performed an exploratory analysis of both datasets.



## AWS Cost Dataset

Observations:

- The dataset contains approximately 90 days of AWS billing information.
- Cost data shows clear day-to-day variability.
- A small number of days exhibit significantly higher costs compared to surrounding periods.
- Cost is strongly correlated with usage quantity.
- Several AWS services contribute to total cost, with a small number of services accounting for the majority of spend.

## Kubernetes Dataset

Observations:

- CPU and memory utilization are relatively stable across most days.
- Pod-level aggregation revealed variability between workloads that is not visible at the cluster level.
- Active pod and active node counts remained almost constant throughout the observation period and therefore provided limited value as predictive features.
- CPU utilization and memory utilization show meaningful relationships with AWS spending patterns.

## Correlation Analysis

A correlation analysis was performed between AWS cost metrics and Kubernetes workload metrics.

Key observations:

- AWS cost is positively correlated with resource utilization.
- CPU and memory utilization provide useful contextual information when explaining cost behaviour.
- Some workload features are highly correlated with one another, therefore only the most informative features were retained for anomaly detection.



Correlation analysis indicates that AWS cost is positively associated with both resource consumption and Kubernetes workload metrics. The strongest relationships were observed between cost, usage quantity, CPU utilization and memory utilization. These findings support the inclusion of Kubernetes telemetry as explanatory features in the anomaly detection model.

---

# 3. Candidate Cost Anomaly Detection

## Methodology

The anomaly detection problem is inherently unsupervised because no labelled anomaly events were provided.

For this reason, I selected:

### Isolation Forest

Reasons:

- Designed specifically for anomaly detection.
  - Handles multivariate feature spaces effectively.
  - Does not require labelled anomalies.
  - Performs well on small and medium-sized datasets.
  - Widely used in operational monitoring and cloud cost anomaly detection.
- 

## Feature Engineering

Features were created from three categories:

### Cost Features

- Total daily cost
- Mean daily cost
- Maximum daily cost
- Cost difference from previous day
- Cost percentage change
- 7-day rolling cost z-score

### Usage Features

- Total usage quantity
- Mean usage quantity

## Kubernetes Workload Features

Pod-level metrics were first aggregated daily and then summarized into daily workload indicators:

- Average CPU utilization
- Maximum CPU utilization
- CPU variability
- Average memory utilization
- Maximum memory utilization
- Memory variability

This approach preserved pod-level workload behaviour while ensuring that all model inputs matched the daily granularity of AWS billing data.

---

## Hyperparameter Stability Analysis

Since no ground truth anomaly labels were available, traditional GridSearchCV could not be used.

Instead, I performed a stability analysis across multiple Isolation Forest configurations:

- `n_estimators` = 100, 200, 300, 500
- `contamination` = 1%, 2%, 3%, 4%, 5%

Multiple models were executed and anomaly dates were counted across all runs.

The final anomalies were selected based on:

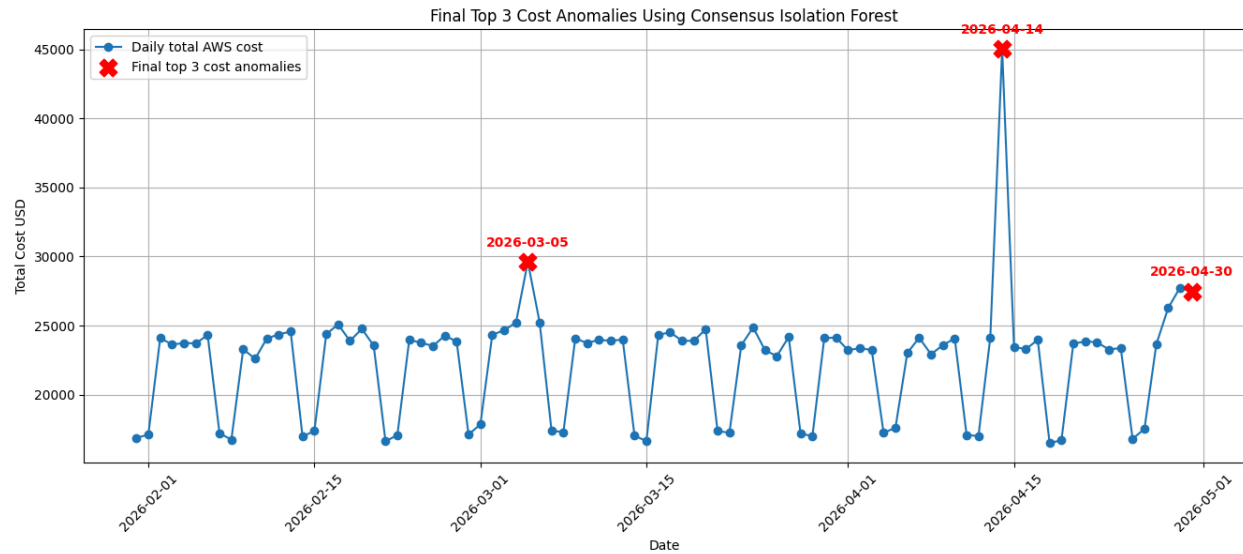
1. Detection stability across multiple configurations.
2. Cost relevance.
3. Magnitude of the anomaly score.

This approach reduces dependency on a single arbitrary hyperparameter selection.

---

# Results

Three candidate cost anomalies were identified.



The selected anomalies consistently appeared across multiple model configurations and showed significant deviations from expected behaviour.

For each anomaly date, root-cause analysis was performed to identify:

- AWS account
- AWS service
- Usage type
- Region

responsible for the largest portion of the cost increase.

---

## 4. Cost Forecasting

### Methodology

For forecasting, I selected:

## Holt-Winters Exponential Smoothing

Reasons:

- The dataset contains only ~90 days of observations.
  - The series exhibits trend behaviour.
  - Holt-Winters performs well on short historical windows.
  - The model is lightweight and easily deployable.
  - Results are highly interpretable for business users.
  - Retraining cost is minimal.
- 

## Forecasting Workflow

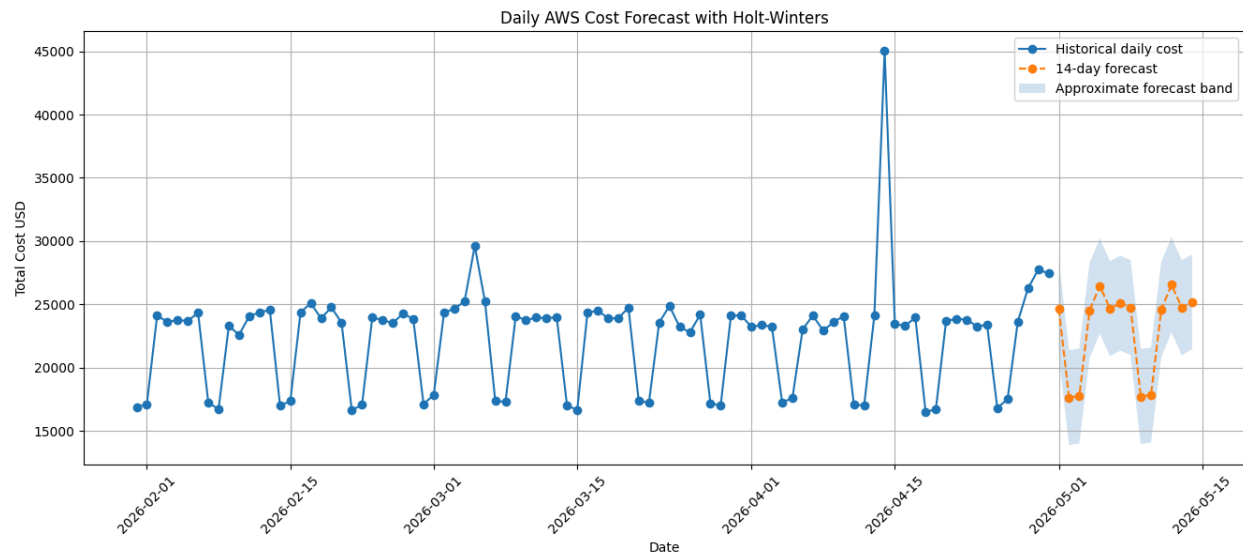
Steps:

1. Aggregate daily AWS cost.
  2. Create train/test split.
  3. Fit Holt-Winters model.
  4. Evaluate forecasting performance.
  5. Generate future cost predictions.
-



## Results

The model successfully captured the underlying cost trend and produced a 14-day forecast horizon.



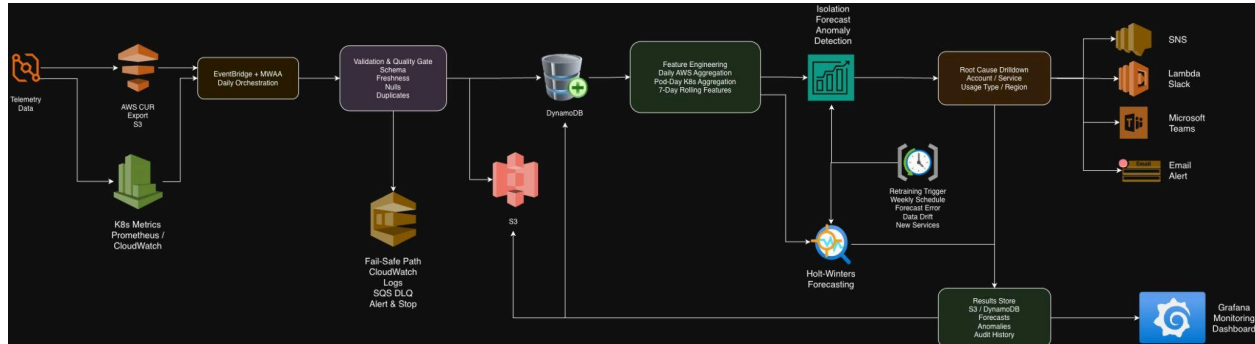
Evaluation metrics such as MAE, RMSE and MAPE were calculated to assess forecasting quality.

The resulting forecast provides an operational estimate of future AWS spending and can be used alongside anomaly detection for proactive cost management.

---

# 5. Proposed Production Architecture

The proposed architecture is designed with scalability, reliability and operational maintainability in mind. It combines AWS-native services for orchestration, monitoring and storage with machine learning components responsible for anomaly detection and forecasting. The solution enables daily automated execution, data quality validation, model retraining triggers, alert delivery and historical result retention, providing a robust foundation for production deployment.



The proposed production solution is designed to run daily and consists of the following components:

## Data Sources

- AWS Cost and Usage Reports (CUR)
- Kubernetes Metrics (CloudWatch / Prometheus)

## Orchestration

- Amazon EventBridge
- Amazon MWAA (Airflow)

## Validation Layer

Validation checks include:

- Schema validation
- Freshness validation
- Missing value validation
- Duplicate detection
- Cost sanity checks

## Processing Layer

Feature engineering jobs generate:

- Daily AWS aggregates
- Daily Kubernetes workload aggregates
- Rolling statistics

## Analytical Layer

Two analytical pipelines are executed:

### Anomaly Detection

- Consensus Isolation Forest

### Forecasting

- Holt-Winters Exponential Smoothing

## Output Layer

Results are stored in:

- Amazon S3
- DynamoDB

and exposed through:

- FastAPI REST service
- Dashboards
- Alerting channels

## Monitoring and Alerting

- CloudWatch Logs
- SNS
- Slack / Teams notifications

## Fail-Safe Mechanisms

- Dead Letter Queue (DLQ)
  - Retry policies
  - Validation failure notifications
  - Fallback to last successful model version
- 

## 6. Information Required Before Deployment

Before deploying the solution into production, I would want answers to the following questions.

### 1. How is AWS cost defined?

It is important to understand whether costs represent:

- Unblended Cost
- Blended Cost
- Amortized Cost
- Net Amortized Cost

Different billing definitions can significantly affect anomaly detection behaviour.

---

### 2. Which operational events are expected?

I would want visibility into:

- Planned deployments
- Infrastructure migrations
- Load testing activities
- Marketing campaigns
- Batch processing jobs

These events can generate legitimate cost spikes and should not trigger unnecessary alerts.

---

### 3. What constitutes a business-critical anomaly?

Technical anomalies and business anomalies are not always identical.

The organization should define:

- Cost increase thresholds
- Budget thresholds
- Service-specific thresholds
- Alert severity levels

This ensures that alerting is aligned with business impact.

---

## 7. Future Improvements

Several enhancements could further improve the solution.

### Short-Term

- Automated model retraining
- Data drift monitoring
- Dynamic anomaly thresholds
- Service-level anomaly detection

### Medium-Term

- Forecast ensembles
- Prophet benchmarking
- Explainability layer for anomalies

### Long-Term

- Near real-time anomaly detection
  - Cost optimization recommendations
  - Integration with FinOps workflows
  - Cloud-native MLOps deployment pipeline
-

# Conclusion

The proposed solution combines machine learning, forecasting, Kubernetes telemetry and cloud cost analytics to provide a practical framework for AWS cost monitoring.

The architecture is production-oriented, containerized, API-enabled and designed with scalability, monitoring and operational robustness in mind.

The resulting platform enables both reactive cost investigation through anomaly detection and proactive planning through forecasting.